

✓ What is a Functional Interface in Java 1.8?

A **Functional Interface** in Java is an interface that contains **exactly one abstract method**.

Java 1.8 introduced **Lambda Expressions**, and functional interfaces are required to use lambdas.

□ In simple words:

If an interface has **only one abstract method**, it is called a *functional interface*.

□ Why Was Functional Interface Introduced in Java 1.8?

In **Java 1.8**, Oracle introduced:

- Lambda Expressions
- Method References
- Stream API

To support **lambda expressions**, Java needed a target type — and that target type is a **functional interface**.

Lambda expressions can only be used where an interface has **one abstract method**.

□ Definition

A functional interface:

- Has **only one abstract method**
 - Can have:
 - Multiple default methods
 - Multiple static methods
 - Can be annotated with `@FunctionalInterface` (optional but recommended)
-

□ Example of Functional Interface

```
@FunctionalInterface
interface MyFunctionalInterface {
    void sayHello();                // Only one abstract method
}
```

❑ Using Functional Interface with Lambda Expression

```
public class Main {  
    public static void main(String[] args) {  
  
        MyFunctionalInterface obj = () -> {  
            System.out.println("Hello from Lambda!");  
        };  
  
        obj.sayHello();  
    }  
}
```

✓ Output:

Hello from Lambda!

Here:

- `() -> {}` is the **lambda expression**
- It implements the `sayHello()` method

❑ Why Is It Called “Functional”?

Because it represents a **single function behavior**.

It focuses on “what to do” rather than “how to create a class”.

Before Java 1.8, we had to write:

```
MyFunctionalInterface obj = new MyFunctionalInterface() {  
    @Override  
    public void sayHello() {  
        System.out.println("Hello from Anonymous Class");  
    }  
};
```

Java 1.8 made it shorter using lambda.

❑ Difference Between Functional Interface and Other Interfaces

Feature	Functional Interface	Normal Interface
Abstract Methods	Only 1	One or more
Lambda Support	✓ Yes	✗ No
Annotation	@FunctionalInterface	Not required
Purpose	Behavior / Function	Contract for multiple methods
Example	Runnable, Comparator	List, Map

❑ Example of Normal Interface

```
interface NormalInterface {  
    void method1();  
    void method2();  
}
```

This is **NOT** a functional interface because it has **two abstract methods**.

You cannot use lambda here.

❑ Built-in Functional Interfaces in Java 8

Java 8 provides many built-in functional interfaces in:

❑ java.util.function package

Interface	Abstract Method	Purpose
Runnable	run()	Thread execution
Callable	call()	Returns value
Comparator	compare()	Compare objects
Predicate<T>	test(T t)	Returns boolean
Function<T,R>	apply(T t)	Takes input, returns output
Consumer<T>	accept(T t)	Takes input, returns nothing

Supplier<T>	get()	Returns value
--------------------------	-------	---------------

□ Example Using Built-in Functional Interface

```
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {

        Predicate<Integer> isEven = (num) -> num % 2 == 0;

        System.out.println(isEven.test(10));           // true
        System.out.println(isEven.test(7));            // false
    }
}
```

□ Important Rules

A functional interface:

- ✓ Must have exactly **one abstract method**
 - ✓ Can have multiple default methods
 - ✓ Can have multiple static methods
 - ✓ Can override Object class methods
-

Example Showing Default + Static Methods

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);                               // Only one abstract method

    default void print() {
        System.out.println("Calculator Interface");
    }

    static void info() {
        System.out.println("Static method inside interface");
    }
}
```

This is still a functional interface because there is only **one abstract method**.

□ Summary

Point	Explanation
What is it?	Interface with exactly one abstract method
Introduced in	Java 8
Why?	To support Lambda expressions
Annotation	@FunctionalInterface (optional)
Main Advantage	Cleaner and shorter code

□ Interview Answer (Short Version)

A functional interface in Java 8 is an interface that contains exactly one abstract method. It enables lambda expressions and method references. Unlike normal interfaces, which can have multiple abstract methods, a functional interface focuses on a single behavior. It may contain multiple default and static methods but only one abstract method.

If you want, I can also explain:

- Functional Interface vs Anonymous Class (deep comparison)
- Real-time industry example
- How Streams use functional interfaces
- Java 8 internal working mechanism

▣ 1 Functional Interface vs Anonymous Class (Deep Comparison)

Before Java 8, we mainly used **Anonymous Inner Classes**.

After Java 8, **Lambda Expressions + Functional Interfaces** became preferred.

□ Basic Concept Difference

Feature	Functional Interface + Lambda	Anonymous Class
Code Length	Very Short	Long & Verbose

Readability	Clean	More boilerplate
Performance	Slightly better (invokedynamic)	Creates separate class
Syntax	() -> {}	new Interface() {}
Best For	Single behavior	Multiple methods / complex logic
Access to this	Refers to outer class	Refers to anonymous class

❑ Example Comparison

✔ Using Anonymous Class (Before Java 8)

```
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {

        Greeting obj = new Greeting() {
            @Override
            public void sayHello() {
                System.out.println("Hello from Anonymous Class");
            }
        };

        obj.sayHello();
    }
}
```

✔ Using Functional Interface + Lambda (Java 8)

```
@FunctionalInterface
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {

        Greeting obj = () ->
            System.out.println("Hello from Lambda");
    }
}
```

```
    obj.sayHello();  
  }  
}
```

☐ Key Internal Difference

Anonymous Class:

- Compiler creates a **separate .class file**
- Uses traditional object creation

Lambda:

- Uses **invokedynamic**
 - No extra class file
 - More optimized at runtime
-

☐ When to Use What?

Scenario	Recommended
Single method behavior	Lambda
Multiple abstract methods	Anonymous class
Complex logic inside	Anonymous class
Simple callback	Lambda

☐ 2 Real-Time Industry Examples

Let's take real-world scenarios.

☐ Example 1: Sorting Employees (Comparator)

In enterprise applications (HR systems, payroll systems), sorting is very common.

Without Lambda (Old Way)

```
Collections.sort(employeeList, new Comparator<Employee>() {  
    public int compare(Employee e1, Employee e2) {
```

```
    return e1.getSalary() - e2.getSalary();  
}  
});
```

With Lambda (Modern Way)

```
employeeList.sort((e1, e2) -> e1.getSalary() - e2.getSalary());
```

✓ Used in:

- Banking systems
 - Payroll systems
 - E-commerce product sorting
-

❑ Example 2: Runnable in Multithreading

Old Way

```
Thread t = new Thread(new Runnable() {  
    public void run() {  
        System.out.println("Thread running");  
    }  
});
```

Java 8 Lambda Way

```
Thread t = new Thread(() ->  
    System.out.println("Thread running"));  
t.start();
```

✓ Used in:

- Microservices
 - Backend APIs
 - Background job processing
-

❑ 3 How Streams Use Functional Interfaces

Java 8 Streams are heavily based on **functional interfaces**.

Streams use:

Stream Method	Functional Interface Used
filter()	Predicate
map()	Function
forEach()	Consumer
reduce()	BinaryOperator
sorted()	Comparator

❑ Practical Example Using Stream

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(10, 15, 20, 25, 30);

        numbers.stream()
            .filter(n -> n % 2 == 0)                // Predicate
            .map(n -> n * 2)                        // Function
            .forEach(n -> System.out.println(n));    // Consumer
    }
}
```

❑ What Happens Internally?

- filter() accepts a **Predicate**
- map() accepts a **Function**
- forEach() accepts a **Consumer**

Each lambda you write is actually implementing a functional interface.

❑ Real Enterprise Stream Example

Imagine an e-commerce system:

```
List<Product> expensiveProducts =
    products.stream()
        .filter(p -> p.getPrice() > 5000)
```

```
.sorted((p1, p2) -> p2.getPrice() - p1.getPrice())  
.collect(Collectors.toList());
```

Used in:

- Flipkart/Amazon-style filtering
 - Banking transaction filtering
 - Analytics dashboards
-

□ Why Streams + Functional Interfaces Are Powerful?

- ✓ Less boilerplate
 - ✓ More readable
 - ✓ Parallel processing support
 - ✓ Declarative programming style
 - ✓ Reduced bugs
-

□ Interview-Level Summary

Functional Interface vs Anonymous Class:

- Lambda is shorter, faster, and cleaner.
- Anonymous class creates a new class file.
- Lambda uses invokedynamic.

Real-Time Usage:

- Sorting (Comparator)
- Filtering (Predicate)
- Multithreading (Runnable)
- Event handling

Streams:

Streams depend entirely on functional interfaces like:

- Predicate
- Function

- Consumer
- Supplier
- Comparator

If you want, I can next explain:

- Internal working of Lambda (Bytecode level)
- Functional Interface memory model
- Method Reference vs Lambda
- Complete Stream lifecycle diagram ☐

Excellent ☑

These five are the **core functional interfaces** in `java.util.function` and are heavily used in **Streams, collections, multithreading, and enterprise applications**.

I'll explain each with:

- ✓ Definition
- ✓ Method Signature
- ✓ When to Use
- ✓ Simple Java Example
- ✓ Real Industry Example

1 Predicate

✓ Definition

A **Predicate** is a functional interface that:

Takes one input and returns a boolean result.

Used mainly for **filtering and condition checking**.

☐ Package: `java.util.function.Predicate`

☐ Abstract Method

```
boolean test(T t);
```

☐ Simple Example

```
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {

        Predicate<Integer> isEven = num -> num % 2 == 0;

        System.out.println(isEven.test(10));           // true
        System.out.println(isEven.test(7));           // false
    }
}
```

❑ Real Industry Example (Filtering Users)

```
List<String> users = Arrays.asList("Anurag", "Admin", "Guest");

Predicate<String> isAdmin = name -> name.equals("Admin");

users.stream()
    .filter(isAdmin)
    .forEach(System.out::println);
```

✓ Used in:

- Authentication systems
 - Transaction validation
 - Fraud detection
-

2 Function<T, R>

✓ Definition

A Function:

Takes one input and returns one output.

Used for **data transformation**.

❑ Package: java.util.function.Function

❑ Abstract Method

R apply(T t);

❑ Simple Example

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {

        Function<Integer, String> convertToString =
            num -> "Number is: " + num;

        System.out.println(convertToString.apply(5));
    }
}
```

❑ Real Industry Example (Salary Calculation)

```
Function<Double, Double> calculateBonus =
    salary -> salary * 1.10;

System.out.println(calculateBonus.apply(50000.0));
```

✓ Used in:

- Data transformation
 - API response mapping
 - DTO conversion
-

3 Consumer

✓ Definition

A Consumer:

Takes one input and returns nothing (void).

Used for **performing actions**.

❑ Package: java.util.function.Consumer

❑ Abstract Method

```
void accept(T t);
```

❑ Simple Example

```
import java.util.function.Consumer;

public class Main {
    public static void main(String[] args) {

        Consumer<String> printName =
            name -> System.out.println("Hello " + name);

        printName.accept("Anurag");
    }
}
```

❑ Real Industry Example (Logging)

```
List<String> logs = Arrays.asList("Login", "Payment", "Logout");

logs.forEach(log -> System.out.println("Event: " + log));
```

✓ Used in:

- Logging systems
 - Sending notifications
 - Writing to database
-

4 Supplier

✓ Definition

A **Supplier**:

Takes no input but returns a value.

Used for **generating or supplying data**.

❑ Package: java.util.function.Supplier

❑ Abstract Method

```
T get();
```

❑ Simple Example

```
import java.util.function.Supplier;

public class Main {
    public static void main(String[] args) {

        Supplier<Double> randomValue =
            () -> Math.random();

        System.out.println(randomValue.get());
    }
}
```

❑ Real Industry Example (OTP Generator)

```
Supplier<Integer> otpGenerator =
    () -> (int)(Math.random() * 10000);

System.out.println("OTP: " + otpGenerator.get());
```

✓ Used in:

- OTP systems
 - Lazy initialization
 - Object creation factories
-

5 Comparator

✓ Definition

A Comparator:

Compares two objects and returns an integer.

❑ Package: java.util.Comparator

❑ Abstract Method

```
int compare(T o1, T o2);
```

Return value meaning:

Return	Meaning
0	Equal
> 0	First greater
< 0	Second greater

□ Simple Example

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(5, 2, 8, 1);

        numbers.sort((a, b) -> a - b);

        System.out.println(numbers);
    }
}
```

□ Real Industry Example (Sort Employees by Salary)

```
class Employee {
    String name;
    int salary;

    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
}

List<Employee> list = Arrays.asList(
    new Employee("A", 50000),
    new Employee("B", 70000)
);

list.sort((e1, e2) -> e2.salary - e1.salary);
```

✓ Used in:

- E-commerce sorting
- HR payroll system
- Banking transaction ranking

□ Summary Table

Interface	Input	Output	Purpose
Predicate	1	boolean	Condition check
Function	1	1	Transform data
Consumer	1	void	Perform action
Supplier	0	1	Provide data
Comparator	2	int	Compare objects

□ How Streams Use Them Together

```
List<Integer> list = Arrays.asList(10, 15, 20, 25);
```

```
list.stream()  
    .filter(n -> n > 15)           // Predicate  
    .map(n -> n * 2)              // Function  
    .forEach(System.out::println); // Consumer
```

□ Interview Summary (Strong Answer)

- Predicate → condition checking
- Function → transformation
- Consumer → action performer
- Supplier → value provider
- Comparator → object comparison

All are **functional interfaces** introduced in Java 8 and form the backbone of the Stream API.

If you want next:

- Memory diagram explanation
- Internal working (bytecode level)
- Real microservice-level example
- Complete CRUD project example using all five

Excellent question ? — this goes deeper into **how lambda works internally in Java 8**.

Let's break it clearly.

1 What is a "Target Type" in Java?

✓ Definition

A **Target Type** is:

The data type (usually a functional interface) that the compiler uses to determine the type of a lambda expression.

In simple words:

- ☐ A lambda expression does NOT have its own type.
- ☐ It gets its type from the context where it is used.

That context type is called the **Target Type**.

☐ Why Does Java Need a Target Type?

Because this is **NOT valid in Java**:

```
() -> System.out.println("Hello");
```

Java doesn't know:

- What method is this implementing?
- What is the return type?
- What are the parameters?

So Java needs a **functional interface reference** to assign the lambda.

☐ Example Without Target Type (✗ Error)

() -> System.out.println("Hello"); // Compile-time error

❑ Example With Target Type (✓ Correct)

```
@FunctionalInterface
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {

        Greeting g = () -> System.out.println("Hello");

        g.sayHello();
    }
}
```

Here:

Greeting is the **Target Type**

The compiler understands:

- Greeting has one method → void sayHello()
- So lambda must match that method

❑ Another Example (Very Important)

```
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {

        Predicate<Integer> isEven = n -> n % 2 == 0;

        System.out.println(isEven.test(10));
    }
}
```

Target Type = Predicate<Integer>

The compiler sees:

```
boolean test(Integer n);
```

So it checks:

- Does lambda return boolean? ✓
- Does it take one Integer parameter? ✓

So it compiles successfully.

☐ **Target Type Can Come From:**

Context	Example
Variable assignment	Predicate<Integer> p = n -> n > 5;
Method parameter	list.sort((a,b) -> a-b);
Return type	return x -> x*x;
Type casting	(Runnable) () -> System.out.println("Hi");

2 What is “Single-Function Behavior”?

✓ Definition

Single-function behavior means:

The interface represents exactly ONE behavior (one action / one task).

A functional interface defines:

- ☐ One abstract method
 - ☐ One behavior
 - ☐ One logical operation
-

☐ **Example 1 – Runnable**

Runnable r = () -> System.out.println("Running task");

What is the behavior?

✓ "Run something"

Only one job: run()

☐ **Example 2 – Predicate**

Predicate<Integer> isAdult = age -> age >= 18;

Single behavior?

✓ "Check condition"

Only one job: test()

☐ **Example 3 – Function**

Function<String, Integer> getLength = str -> str.length();

Single behavior?

✓ "Transform input to output"

Only one job: apply()

☐ **Why Java Requires Single-Function Behavior?**

Because lambda represents:

- ☐ A single block of logic
- ☐ A single implementation

If an interface had multiple abstract methods:

```
interface Example {  
    void method1();  
    void method2();  
}
```

Then this would be confusing:

Example e = () -> ??? // Which method?

Java wouldn't know which method the lambda is implementing.

That's why:

- ✓ Functional Interface = Exactly 1 abstract method
 - ✓ So lambda maps clearly to that one method
-

□ Real-World Analogy

Think of it like this:

Functional Interface = One Button Machine

You press button → One action happens.

Normal Interface = Multiple Button Machine

You press button → Many options possible.

Lambda works only when there is ONE button.

□ Deep Technical Understanding

Lambda expression internally:

- Does NOT create a new anonymous class
 - Uses invokedynamic
 - Binds dynamically to a functional interface
 - Needs a target type for type inference
-

□ Final Summary

✓ Target Type

- The functional interface type that provides context to a lambda expression.
 - Without it, lambda cannot compile.
 - Comes from assignment, method parameter, or return type.
-

✓ Single-Function Behavior

- Means one abstract method.
 - Represents one logical operation.
 - Allows lambda to map clearly to that method.
-

□ Interview-Level Answer

Target type is the functional interface type that provides context to a lambda expression. Since lambda expressions do not have their own type, the compiler uses the target type to determine which abstract method the lambda implements.

Single-function behavior means the interface defines exactly one abstract method, representing one logical operation, which allows lambda expressions to implement it unambiguously.

If you want next:

- Type inference deep explanation
- invokedynamic internal JVM working
- Memory model difference between lambda and anonymous class
- Bytecode comparison

✓ Meaning of “Single Function Behavior” in Java

□ Simple Definition

Single-function behavior means an interface represents **only one logical operation (one behavior)** through **exactly one abstract method**.

□ Break It Down

- **Single** → Only one
- **Function** → Method (operation)
- **Behavior** → What the method does

□ So:

A functional interface defines **one action / one responsibility / one behavior**

❑ **Example 1: Runnable**

```
Runnable r = () -> System.out.println("Task is running");
```

What is the behavior?

✓“Run a task”

Only one method exists:

```
void run();
```

❑ This interface represents **one behavior only** → **running a task**

❑ **Example 2: Predicate**

```
Predicate<Integer> isEven = n -> n % 2 == 0;
```

Behavior?

✓“Check condition”

Only one method:

```
boolean test(T t);
```

❑ One behavior → return true/false

❑ **Example 3: Function**

```
Function<String, Integer> length = s -> s.length();
```

Behavior?

✓“Transform input to output”

Only one method:

```
R apply(T t);
```

❑ One behavior → conversion

✗ What is NOT Single-Function Behavior?

```
interface Example {  
    void method1();  
    void method2();  
}
```

□ This has **multiple behaviors**:

- method1 → behavior 1
- method2 → behavior 2

So this is **NOT a functional interface**

□ Why Single-Function Behavior is Important

Because of **Lambda Expressions**

Example `e = () -> ???` // ✗Confusion

□ Which method should lambda implement?

Java cannot decide.

✓ With Single Behavior (Works)

```
interface Greeting {  
    void sayHello();  
}
```

`Greeting g = () -> System.out.println("Hello");`

✓ No confusion

✓ Lambda maps directly to one method

□ Real-World Analogy

□ Functional Interface (Single Behavior)

Think of a **remote with only ONE button**

☐ Press → One action happens

☐ **Normal Interface (Multiple Behaviors)**

Think of a **remote with MANY buttons**

☐ Which button does lambda represent? ✗

☐ **Key Insight**

Lambda = implementation of **one method only**

So Java requires:

- ✓ One abstract method
 - ✓ One behavior
 - ✓ One clear mapping
-

☐ **Interview Answer**

Single-function behavior means that a functional interface represents exactly one logical operation through a single abstract method. This allows lambda expressions to provide a direct and unambiguous implementation of that method.

If you want, I can go deeper into:

- How JVM maps lambda to that single method
- Bytecode-level explanation
- Real Spring Boot examples using this concept ☐

Let's break this down clearly and practically so you can *actually use generics in real projects*.

✓ 1. What is a Generic Type in Java?

A **generic type** allows you to define a class, interface, or method with a **placeholder type (like <T>)**, so it can work with **any data type safely**.

□ **Simple definition:**

Generics = **Type Safety + Reusability + No Casting**

□ **Example:**

```
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}
```

Usage:

```
Box<String> box = new Box<>();
box.set("Hello");
String data = box.get(); // No casting needed
```

□ **2. Java Before vs After Generics**

✗ **Before Generics (Java < 5)**

- Everything stored as Object
- **Manual casting required**
- **Runtime errors (ClassCastException)**

```
import java.util.*;
```

```
public class OldExample {
    public static void main(String[] args) {
        List list = new ArrayList(); // No type safety

        list.add("Hello");
        list.add(100); // Allowed (problem!)

        String s = (String) list.get(1); // Runtime error ✗
    }
}
```

❑ Problems:

- No compile-time checking
 - Casting everywhere
 - Bugs appear at runtime
-

✔ After Generics (Java 5+)

- Type safety at compile time
- No casting needed
- Cleaner & safer code

```
import java.util.*;

public class NewExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();

        list.add("Hello");
        // list.add(100);      ✗ Compile-time error

        String s = list.get(0);    // No casting needed
    }
}
```

❑ Benefits:

- Compile-time error detection
 - Cleaner code
 - Better readability
-

❑ 3. Generics in Java Collections (Real-Time Use Cases)

Java Collections heavily use generics.

✔ Example 1: Employee Management System

```
import java.util.*;

class Employee {
    int id;
    String name;
```

```

Employee(int id, String name) {
    this.id = id;
    this.name = name;
}
}

public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(1, "Anurag"));
        employees.add(new Employee(2, "Rahul"));

        for (Employee emp : employees) {
            System.out.println(emp.name);
        }
    }
}

```

□ Real-world use:

- HR systems
- Payroll systems
- User data handling

✓ Example 2: Map for Caching (Key-Value)

```

import java.util.*;

public class CacheExample {
    public static void main(String[] args) {
        Map<Integer, String> cache = new HashMap<>();

        cache.put(1, "User1");
        cache.put(2, "User2");

        System.out.println(cache.get(1));
    }
}

```

□ Real-world use:

- Caching systems
- Database result mapping
- API responses

✔ Example 3: Queue in Order Processing

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<String> orders = new LinkedList<>();

        orders.add("Order1");
        orders.add("Order2");

        while (!orders.isEmpty()) {
            System.out.println("Processing: " + orders.poll());
        }
    }
}
```

□ Real-world use:

- E-commerce order queues
- Task scheduling
- Messaging systems

□ 4. Generics Without Collections (Real-Time Use Cases)

Generics are even more powerful when used in **custom classes and methods**.

✔ Example 1: Generic API Response Wrapper

```
class ApiResponse<T> {
    private T data;
    private String message;

    public ApiResponse(T data, String message) {
        this.data = data;
        this.message = message;
    }

    public T getData() {
        return data;
    }
}
```

Usage:

```
ApiResponse<String> response = new ApiResponse<>("Success", "OK");
ApiResponse<Integer> code = new ApiResponse<>(200, "Status Code");
```

□ Real-world use:

- REST APIs
 - Spring Boot responses
 - Microservices
-

✓ Example 2: Generic Utility Method

```
public class Utility {

    public static <T> void printArray(T[] array) {
        for (T item : array) {
            System.out.println(item);
        }
    }

    public static void main(String[] args) {
        Integer[] nums = {1, 2, 3};
        String[] names = {"A", "B"};

        printArray(nums);
        printArray(names);
    }
}
```

□ Real-world use:

- Utility libraries
 - Logging frameworks
 - Data processing
-

✓ Example 3: Generic Repository (Used in Spring Boot)

```
class Repository<T> {

    public void save(T entity) {
        System.out.println("Saving: " + entity);
    }
}
```

Usage:

```
Repository<String> repo1 = new Repository<>();  
repo1.save("User Data");
```

```
Repository<Integer> repo2 = new Repository<>();  
repo2.save(100);
```

❑ Real-world use:

- Database abstraction layer
- DAO pattern
- Spring Data JPA

❑ **Key Concepts You Must Remember**

❑ **Common Generic Syntax**

Concept	Example
Generic Class	class Box<T>
Generic Method	<T> void print(T t)
Multiple Types	<K, V>
Bounded Types	<T extends Number>

❑ **Why Generics Matter**

- ✓ Compile-time type safety
- ✓ No casting
- ✓ Reusable code
- ✓ Cleaner APIs
- ✓ Backbone of Java Collections

❑ **Final Summary**

- Before generics → unsafe + casting + runtime errors
 - After generics → safe + clean + compile-time checking
 - Collections → heavily depend on generics
 - Real-world → APIs, DB layers, queues, caching, utilities
-

If you want next level ☐ I can show:

- Wildcards (? extends, ? super)
- Generic inheritance
- How Spring Boot uses generics internally
- Interview questions on generics (very important ☐)

Great—this is where Java generics become *powerful and interview-critical*. Let's go step by step with clear intuition + real-world usage.

☐ 1. Wildcards in Java (? extends, ? super)

Wildcards are used when you **don't know the exact type**, but want flexibility.

✓ 1.1 ? extends T → Upper Bound (READ-ONLY)

☐ Means:

“Accept T or any subclass of T”

☐ **Rule:**

- ✓ Can **read**
 - ✗ Cannot safely **add**
-

Example:

```
import java.util.*;

public class ExtendsExample {
    public static void printNumbers(List<? extends Number> list) {
        for (Number n : list) {
            System.out.println(n);
        }

        // list.add(10); ✗ Not allowed
    }
}
```

```

public static void main(String[] args) {
    List<Integer> ints = Arrays.asList(1, 2, 3);
    List<Double> doubles = Arrays.asList(1.1, 2.2);

    printNumbers(ints);
    printNumbers(doubles);
}
}

```

□ Real-world:

- Reading data from APIs
- Processing lists (analytics, reporting)

✓ 1.2 ? super T → Lower Bound (WRITE-ONLY)

□ Means:

“Accept T or any superclass of T”

□ **Rule:**

- ✓ Can **add**
- □ Reading gives Object

Example:

```

import java.util.*;

public class SuperExample {
    public static void addNumbers(List<? super Integer> list) {
        list.add(10);
        list.add(20);
    }

    public static void main(String[] args) {
        List<Number> nums = new ArrayList<>();
        List<Object> objs = new ArrayList<>();

        addNumbers(nums);
        addNumbers(objs);
    }
}

```

□ Real-world:

- Writing into collections
 - Bulk insert systems
-

☐ **Golden Rule (VERY IMPORTANT)**

PECS Rule

- ☐ **Producer** → extends (read)
 - ☐ **Consumer** → super (write)
-

☐ **2. Generic Inheritance**

Generics behave differently than normal inheritance.

✗ Important Rule:

List<Integer> ✗ is NOT a subtype of List<Number>

Even though:

Integer extends Number ✓

✓ Correct Way → Use Wildcards

List<? extends Number> list;

☐ **Example: Generic Parent Class**

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

```
class Cage<T extends Animal> {
```

```

private T animal;

public void set(T animal) {
    this.animal = animal;
}

public void makeSound() {
    animal.sound();
}
}

```

Usage:

```

public class Main {
    public static void main(String[] args) {
        Cage<Dog> dogCage = new Cage<>();
        dogCage.set(new Dog());
        dogCage.makeSound();
    }
}

```

❑ Generic Method with Inheritance

```

public static void printAnimals(List<? extends Animal> animals) {
    for (Animal a : animals) {
        a.sound();
    }
}

```

❑ 3. How Spring Boot Uses Generics Internally

Spring Boot uses generics **everywhere** for flexibility and abstraction.

✔ 3.1 Repository Layer (Spring Data JPA)

```

public interface JpaRepository<T, ID> extends PagingAndSortingRepository<T, ID> {
}

```

Usage:

```

@Entity
class User {
    Long id;
    String name;
}

```

```
public interface UserRepository extends JpaRepository<User, Long> {  
}
```

□ What generics do:

- $T \rightarrow$ Entity type (User)
- $ID \rightarrow$ Primary key (Long)

✓ No need to write CRUD manually

✓ 3.2 ResponseEntity (API Responses)

```
public class ResponseEntity<T> {  
    private T body;  
}
```

Usage:

```
@GetMapping("/user")  
public ResponseEntity<User> getUser() {  
    return ResponseEntity.ok(new User());  
}
```

□ Real-world:

- Type-safe API responses
 - Works for any object (User, Order, etc.)
-

✓ 3.3 RestTemplate / WebClient

```
ResponseEntity<String> response =  
    restTemplate.getForEntity(url, String.class);
```

□ Generic allows:

- Dynamic response type
 - JSON $\rightarrow$ Object mapping
-

✓ 3.4 Generic Service Layer (Common Pattern)

```
public interface BaseService<T> {
```

```
T save(T entity);
List<T> findAll();
}
```

Implementation:

```
public class UserService implements BaseService<User> {
    public User save(User user) {
        return user;
    }

    public List<User> findAll() {
        return List.of(new User());
    }
}
```

☐ Real-world:

- Reusable business logic
- Clean architecture

✔ 3.5 Spring Dependency Injection + Generics

Spring can inject based on generic types:

```
@Autowired
private List<MyService> services;
```

Or:

```
@Component
class Processor<T> { }
```

☐ Used in:

- Strategy patterns
- Plugin systems

☐ **Final Mental Model**

☐ **Wildcards**

Type	Use	Memory Trick
------	-----	--------------

? extends T	Read	Producer
? super T	Write	Consumer

□ Generic Inheritance

- List<Integer> ≠ List<Number>
- Use wildcards to solve it

□ Spring Boot

- Repository → <T, ID>
- API → ResponseEntity<T>
- Services → BaseService<T>
- HTTP → Dynamic response mapping

□ If You Want Next Level

I can show you:

- Type erasure (VERY important interview topic)
- Wildcards deep dive with tricky cases
- Real Spring Boot project using generics
- Advanced generics (Comparable, Comparator, etc.)

Perfect—this is the **advanced layer** where most developers get confused (and where interviews love to go deep). I'll give you **clear rules + tricky cases + real-world patterns**.

□ PART 1: Wildcards Deep Dive (Tricky Cases)

□ 1. Why List<?> is NOT useful for adding

```
List<?> list = new ArrayList<String>();
```

```
list.add("Hello");    ✗ Not allowed
list.add(null);       ✓ Only null allowed
```

□ Why?

- Compiler doesn't know the exact type (?)
- Could be List<Integer>, List<Double>, etc.

□ So:

List<?> = **read-only (almost)**

□ 2. Difference Between <?> vs <Object>

```
List<Object> list1 = new ArrayList<String>();    ✗ Not allowed  
List<?> list2 = new ArrayList<String>();       ✓ Allowed
```

□ Key idea:

- List<Object> = strict type
 - List<?> = unknown type (flexible)
-

□ 3. Tricky: extends prevents adding

```
List<? extends Number> list = new ArrayList<Integer>();
```

```
// list.add(10);    ✗  
// list.add(3.14); ✗
```

□ WHY?

Because actual type might be:

- List<Integer>
- List<Double>

Compiler cannot guarantee safety.

□ 4. Tricky: Reading from super

```
List<? super Integer> list = new ArrayList<Number>();
```

```
list.add(10);           // ✓
```

```
Object obj = list.get(0); // Only Object !
```

- You lose type information when reading.
-

□ 5. Wildcard Capture Problem

```
public static void print(List<?> list) {  
    // list.set(0, list.get(0));    ✗ Compile error  
}
```

- Fix using helper method:

```
public static void print(List<?> list) {  
    helper(list);  
}  
  
private static <T> void helper(List<T> list) {  
    list.set(0, list.get(0));    // ✓  
}
```

- This is called **Wildcard Capture**
-

□ 6. Combining extends + super (Real tricky)

```
public static void copy(List<? extends Number> src,  
                       List<? super Number> dest) {  
    for (Number n : src) {  
        dest.add(n);  
    }  
}
```

- Real-world:

- Copy data safely between collections
-

□ 7. Method Overloading with Wildcards (Danger)

```
void process(List<String> list) {}  
void process(List<Integer> list) {}    ✗ Compile error
```

- Why?

Type erasure makes both:

```
process(List list)
```

❑ PART 2: Advanced Generics

✔ 1. Comparable (Natural Sorting)

- ❑ Used for **default sorting**

```
class User implements Comparable<User> {
    int age;

    User(int age) {
        this.age = age;
    }

    @Override
    public int compareTo(User other) {
        return this.age - other.age;
    }
}
```

Usage:

```
List<User> list = new ArrayList<>();
list.add(new User(30));
list.add(new User(20));
```

```
Collections.sort(list);           // Uses compareTo
```

❑ Real-world:

- Sorting users by age
 - Sorting products by price
 - Ranking systems
-

✔ 2. Comparator (Custom Sorting)

- ❑ Used for **multiple sorting strategies**

```
import java.util.*;
```

```
class User {
    int age;
    String name;
```

```

    User(int age, String name) {
        this.age = age;
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        List<User> list = new ArrayList<>();

        list.add(new User(30, "Anurag"));
        list.add(new User(20, "Rahul"));

        // Sort by age
        list.sort((u1, u2) -> u1.age - u2.age);

        // Sort by name
        list.sort((u1, u2) -> u1.name.compareTo(u2.name));
    }
}

```

❑ Real-world:

- Sort by price, rating, name
 - Multi-column sorting (DB-like)
-

❑ Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo()	compare()
Sorting	Default	Custom
Modify class	Yes	No

✔ 3. Bounded Generics (<T extends ...>)

```

class Calculator<T extends Number> {
    public double add(T a, T b) {
        return a.doubleValue() + b.doubleValue();
    }
}

```

```
}  
}
```

☐ Only accepts:

- Integer
 - Double
 - Float
 - etc.
-

✔ 4. Multiple Bounds

```
class Example<T extends Number & Comparable<T>> {  
}
```

☐ T must:

- Extend Number
 - Implement Comparable
-

✔ 5. Generic Method (Advanced)

```
public class Util {  
  
    public static <T extends Comparable<T>> T max(T a, T b) {  
        return (a.compareTo(b) > 0) ? a : b;  
    }  
}
```

Usage:

```
int result = Util.max(10, 20);  
String s = Util.max("A", "B");
```

✔ 6. PECS in Real Systems

Example: Payment Processing

```
class Payment {}  
class UpiPayment extends Payment {}  
class CardPayment extends Payment {}
```

```

public void processPayments(List<? extends Payment> payments) {
    for (Payment p : payments) {
        System.out.println("Processing...");
    }
}
public void addPayments(List<? super Payment> payments) {
    payments.add(new UpiPayment());
}

```

☐ 7. Type Erasure (VERY IMPORTANT)

☐ At runtime:

List<String> → List
List<Integer> → List

☐ Generics exist only at compile time.

☐ 8. Cannot Do These

new T();	✗
T.class	✗
new List<String>[]	✗

☐ 9. Real Interview Trick Question

```
List<? extends Number> list = new ArrayList<Integer>();
```

```

Number n = list.get(0);    // ✓
list.add(10);              // ✗

```

☐ WHY?

- Safe to read
 - Not safe to write
-

☐ FINAL MASTER RULES

☐ Wildcards Summary

Type	Read	Write	Use Case
<?>	Object	✗	Unknown
? extends T	T	✗	Read
? super T	Object	✓	Write

❑ Advanced Generics Summary

- Comparable → default sorting
- Comparator → custom sorting
- <T extends Number> → restrict types
- Type erasure → runtime removal
- PECS → Producer Extends, Consumer Super

❑ If You Want Next Level (Highly Recommended)

I can show you:

- ✓ Real Spring Boot project using generics (CRUD + API)
- ✓ How Comparator.comparing() works internally
- ✓ Advanced wildcard interview problems (hard level)
- ✓ How Java Streams use generics internally

Target Type in Java

The context where a lambda expression or method reference is used determines the **target type**.

Assigning to a Functional Interface:

```
Runnable r = () →  
    System.out.println("Hello");
```

Target Type: **Runnable**

Passing as a Method Argument:

```
list.forEach(s → )  
    System.out.println(s);
```

Target Type: **Consumer<String>**

???

Return from a Method:

```
return (a, b) → a + b;
```

Target Type: **BiFunction<Integer, Integer, Integer>**

Used in a Conditional Expression:

```
Predicate<String> p = flag ?  
    s → s.isEmpty() : String::isBlank;
```

Target Type: **Predicate<String>**

The target type is the expected type that the lambda expression or method reference is compatible with.

It determines **how the lambda** is interpreted.



✓ Meaning of Target Type in Java

In Java, **target type** means:

- **The expected type of an expression based on the context in which it is used.**
-

□ Simple Explanation

When you write code (especially **lambda expressions** or **method references**), Java does not always know the type directly.

So it looks at the **context (where the expression is used)** to determine the type — this is called the **target type**.

□ Key Idea

- Java is **context-aware**
 - The **surrounding code decides the type**
 - Mostly used in:
 - Lambda expressions
 - Method references
 - Generics
-

□ Examples

1. Lambda with Assignment

```
Runnable r = () -> System.out.println("Hello");
```

- Target type = Runnable
Because the lambda is assigned to a Runnable variable.
-

2. Lambda as Method Argument

```
list.forEach(s -> System.out.println(s));
```

- Target type = Consumer<String>
Because `forEach()` expects a Consumer.
-

3. Return from Method

return (a, b) -> a + b;

□ Target type = BiFunction<Integer, Integer, Integer>
Because the method return type defines it.

4. Conditional Expression

Predicate<String> p = flag ? s -> s.isEmpty() : String::isBlank;

□ Target type = Predicate<String>

⚡ One-Line Definition

Target type = the type that Java expects in a given context, used to interpret expressions like lambdas.

□ Why It Matters

- Helps Java **infer types automatically**
 - Reduces need for **explicit type declarations**
 - Makes code **cleaner and shorter**
-

If you want, I can give you **interview questions + tricky examples** on target types (very useful for Java developer roles).